

Deep Learning

Lecture 7: Flow Models and Generalisation

Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

Durham University





1 Generative Models

- Learning the distribution
- Probability examples
- Max likelihood estimation

2 Flow Models

3 Generalisation

- Universal approximation
- Width and depth?
- Noise, training and regularisation
- No free lunch
- Bias-variance tradeoff
- Double descent

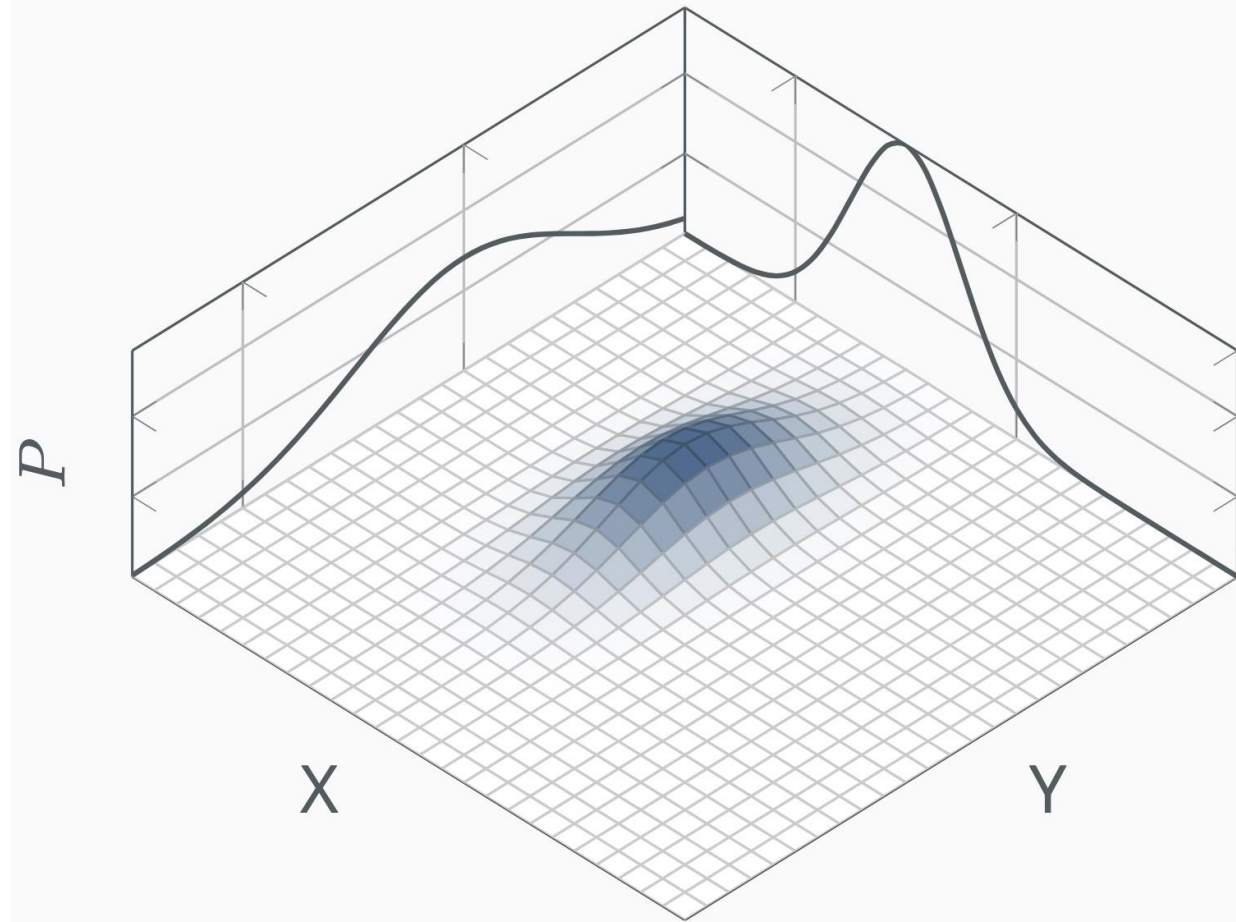
Learning the data distribution

So what is it we want exactly?

- $P(Y|X)$ discriminative model (classification)
- $P(X|Y)$ conditional generative model
- $P(X, Y)$ generative model

We want to learn the probability density function of our data (nature's distribution)

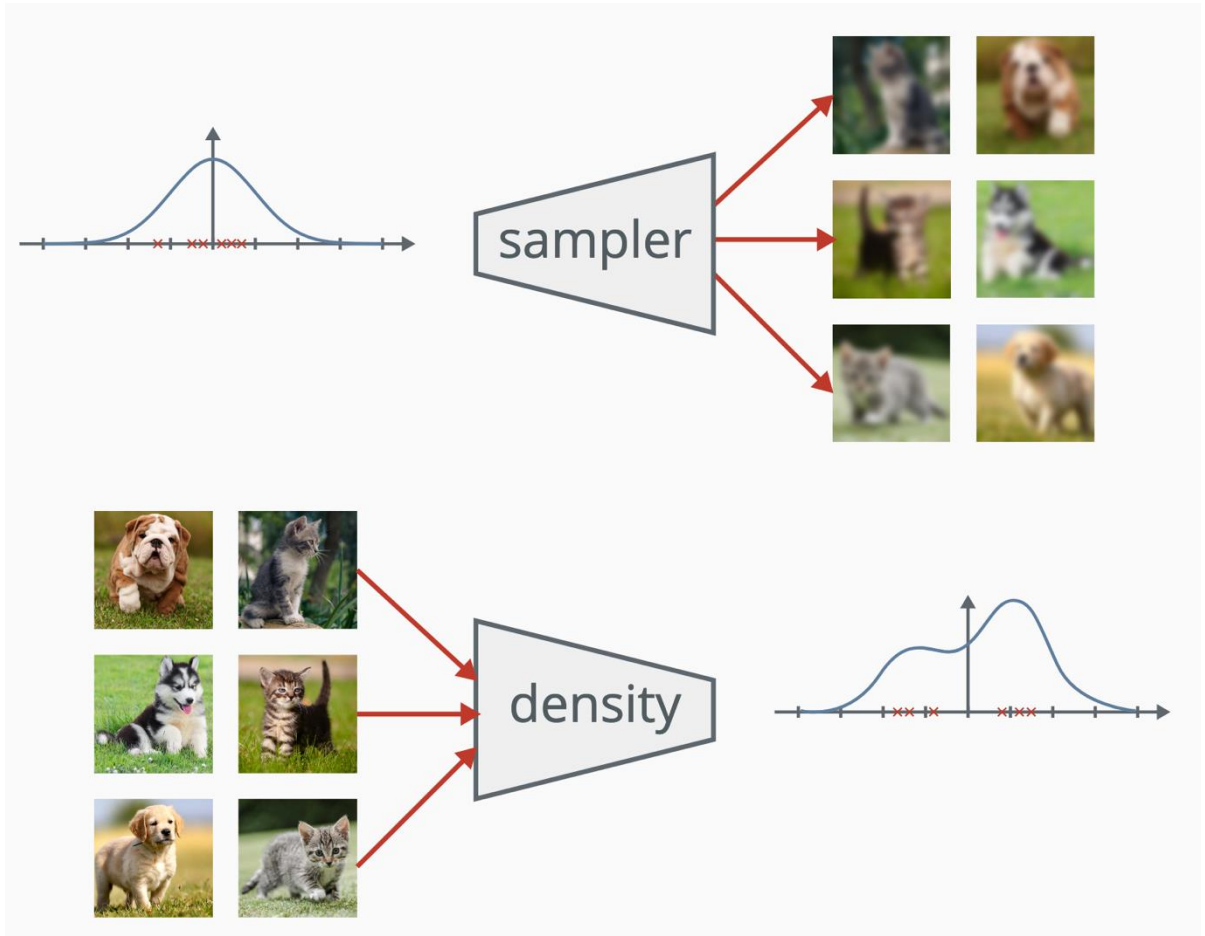
The data distribution $P(X, Y)$



Definition: Generative models learn a joint distribution over the entire dataset. They are mostly used for sampling applications or density estimation:

Sampling the model

A generative model learns to fit a model distribution over observations so we can sample novel data from the model distribution, $\mathbf{x}_{\text{new}} \sim p_{\text{model}}(\mathbf{x})$



Density estimation

Density estimation is estimating the probability of observations. Given a datapoint $\mathbf{x}$, what is the probability assigned by the model, $p_{\text{model}}(\mathbf{x})$?

Probability Examples



Examples

Linguists

- What is the probability of a sentence? $P(\text{sentence})$
 - $P(\text{'the dog chased after the ball'})$
 - $P(\text{'printers eat avocados when sad'}) \approx 0$

Meteorologists

- What is the probability of whether it will rain? $P(\text{rain})$

Artists

- What is the probability of this image being a face? $P(\text{face})$

Musicians

- What is the probability this sounds like Beethoven? $P(\text{Beethoven})$

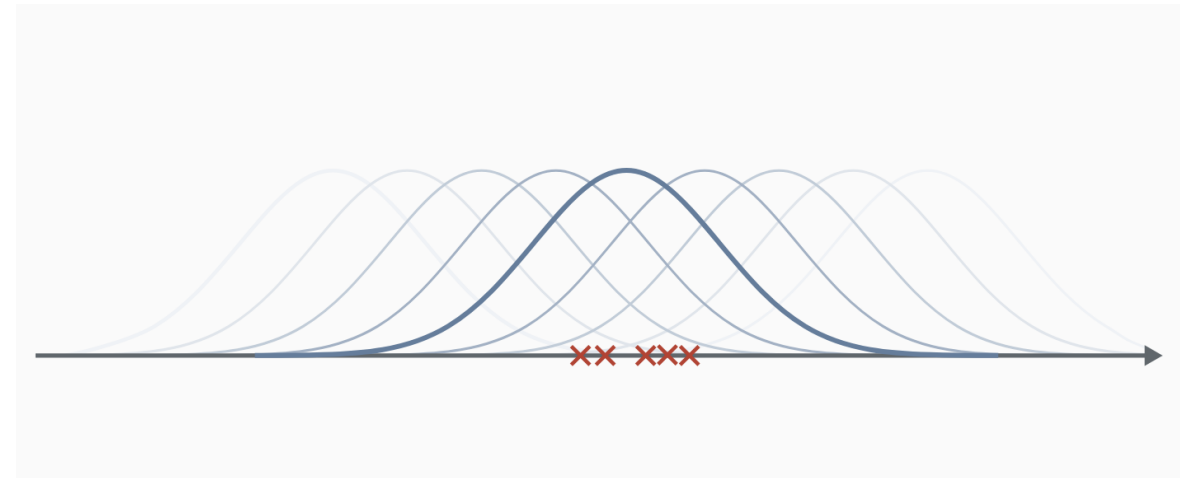


Definition: maximum likelihood estimation

Maximum likelihood estimation (MLE) is a method for estimating the parameters of a probability distribution by maximizing a likelihood function, so that under the model the observed data is most probable

$$\begin{aligned}\theta^* &= \arg \max_{\theta} p_{\text{model}}(\mathbb{X}; \theta) \\ &= \arg \max_{\theta} \prod_{i=1}^n p_{\text{model}}(\mathbf{x}^i; \theta) \\ &\approx \arg \max_{\theta} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_{\text{model}}(\mathbf{x}; \theta)],\end{aligned}$$

where $\mathbb{X} = \{\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^n\}$ are from $p_{\text{data}}(\mathbf{x})$



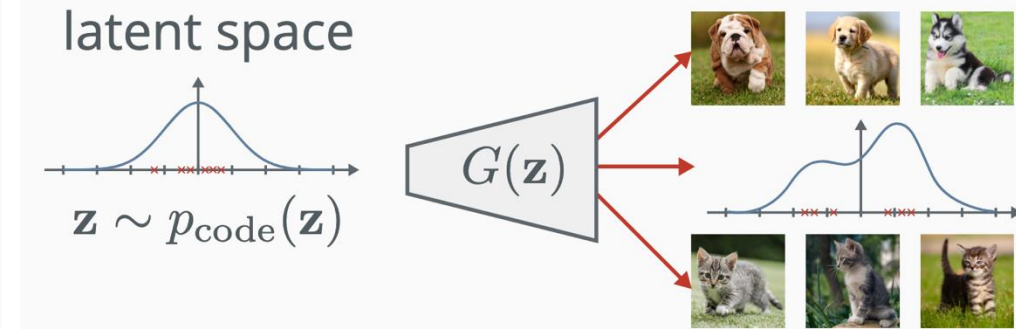
Generative Models



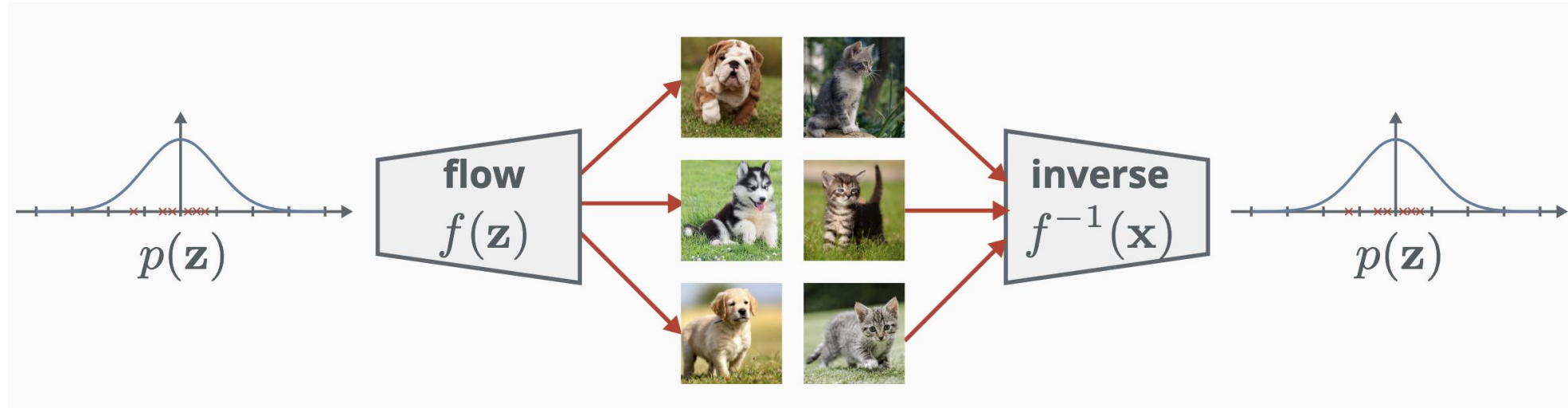
Definition: generative networks

The goal of generative networks is to take some simple distribution, like a normal distribution or a uniform distribution, and apply a non-linear transformation (e.g. a deep neural network) to obtain samples from $p_{\text{data}}(\mathbf{x})$

In 1D, we can say $G = F_{\text{data}}^{-1}(\mathbf{x})$ and sample $\mathbf{z} \sim U$, and similarly in ND — but assuming the determinant of the Jacobian and the inverse of G are computable, which is a large restriction. Ideally we want $\mathbf{z}$ in low dimensions



Flow models restrict our function to be a chain of invertible functions, called a **flow**, therefore the whole function is invertible!

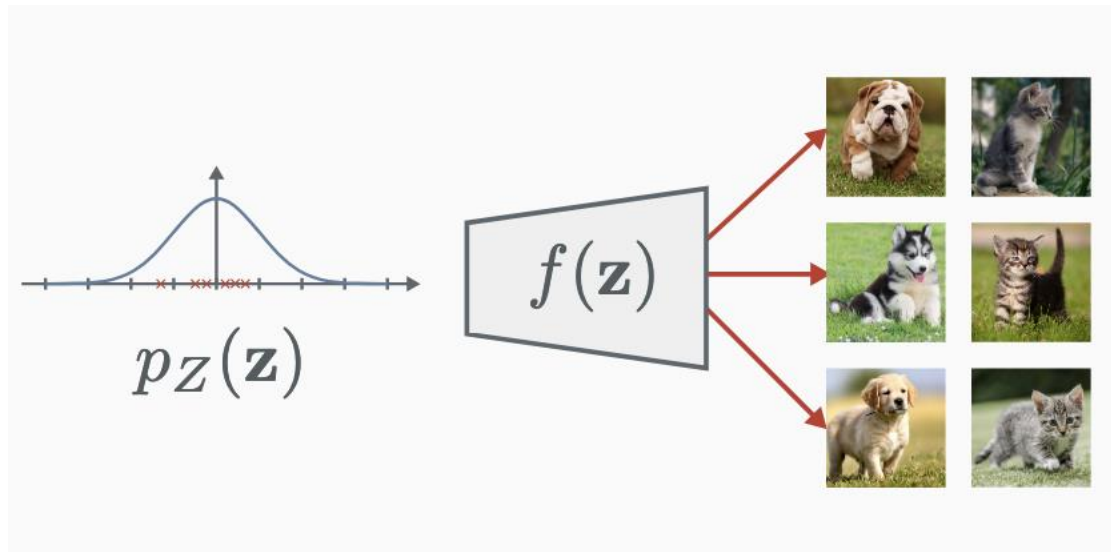


Basically, a *deep generative neural network that can generate data and can be inverted to give us the exact likelihood of the data!*

How do we get the exact likelihood of data?

Given $p_Z(\mathbf{z})$ where $\mathbf{x} = f(\mathbf{z})$ and $\mathbf{z} = f^{-1}(\mathbf{x})$, we ask what is $p_X(\mathbf{x})$?

$$p_X(\mathbf{x}) = p_Z(f^{-1}(\mathbf{x})) \left| \det \left(\frac{\partial f^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$



Normalising Flows



Normalising flows transform and renormalise a sample through a chain of bijective transformations!

Flow models must satisfy the following conditions:

- It must be bijective, ensuring a one-to-one mapping between inputs and outputs.
- The Jacobian determinant must be non-zero, guaranteeing local invertibility.
- The transformation must be simple and structured in a way that allows for easy inversion, often through specific architectural choices like affine transformations, coupling layers, and invertible convolutions.

Normalising Flows

Normalising flow layers



Description	Function	Log-Determinant
Additive Coupling [1]	$\mathbf{y}^{(1:d)} = \mathbf{x}^{(1:d)}$ $\mathbf{y}^{(d+1:D)} = \mathbf{x}^{(d+1:D)} + f(\mathbf{x}^{(1:d)})$	0
Planar [2]	$\mathbf{y} = \mathbf{x} + \mathbf{u}h(\mathbf{w}^T \mathbf{z} + b)$ With $\mathbf{w} \in \mathbb{R}^D, \mathbf{u} \in \mathbb{R}^D, \mathbf{b} \in \mathbb{R}$	$\ln 1 + \mathbf{u}^T h'(\mathbf{w}^T \mathbf{z} + b)\mathbf{w} $
Affine Coupling [3]	$\mathbf{y}^{(1:d)} = \mathbf{x}^{(1:d)}$ $\mathbf{y}^{(d+1:D)} = \mathbf{x}^{(d+1:D)} \odot f_\sigma(\mathbf{x}^{(1:d)}) + f_\mu(\mathbf{x}^{(1:d)})$	$\sum_1^d \ln f_\sigma(x^{(i)}) $
Batch Normalization [3]	$\mathbf{y} = \frac{\mathbf{x} - \tilde{\mu}}{\sqrt{\tilde{\sigma}^2 + \epsilon}}$	$-\frac{1}{2} \sum_i \ln(\tilde{\sigma}_i^2 + \epsilon)$
1x1 Convolution [4]	With $h \times w \times c$ tensor $\mathbf{x}$ & $c \times c$ tensor $\mathbf{W}$ $\forall i, j : \mathbf{y}_{i,j} = \mathbf{W} \mathbf{x}_{i,j}$	$h \cdot w \cdot \ln \det \mathbf{W} $
i-ResNet [5]	$\mathbf{y} = \mathbf{x} + f(\mathbf{x})$ where $\ f\ _L < 1$	$\text{tr}(\ln(\mathbf{I} + \nabla_{\mathbf{x}} f)) =$ $\sum_{k=1}^{\infty} (-1)^{k+1} \frac{\text{tr}((\nabla_{\mathbf{x}} f)^k)}{k}$
Emerging Convolutions [6]	$\mathbf{k} = \mathbf{w}_1 \odot \mathbf{m}_1, \quad \mathbf{g} = \mathbf{w}_2 \odot \mathbf{m}_2$ $\mathbf{y} = \mathbf{k} \star_l (\mathbf{g} \star_l \mathbf{x})$	$\sum_c \ln \mathbf{k}_{c,c,m_y,m_x} \mathbf{g}_{c,c,m_y,m_x} $

[1] Dinh et al., 2014

[2] Danilo et al., 2015

[3] Dinh et al., 2016

[4] Kingma et al., 2018

[5] Behrmann et al., 2019

[6] Hoogetboom et al., 2019

Flow Models



Computing the determinants can be expensive, so the focus is on restricting the function f^{-1} to those with easy-to-compute Jacobian determinants.

- Only using invertible functions is very restrictive!
- Large number of transformations/layers are needed to make these work.
- They are not quite as expressive as needed.



[Kingma and Dhariwal, 2018]

Generalisation Theory

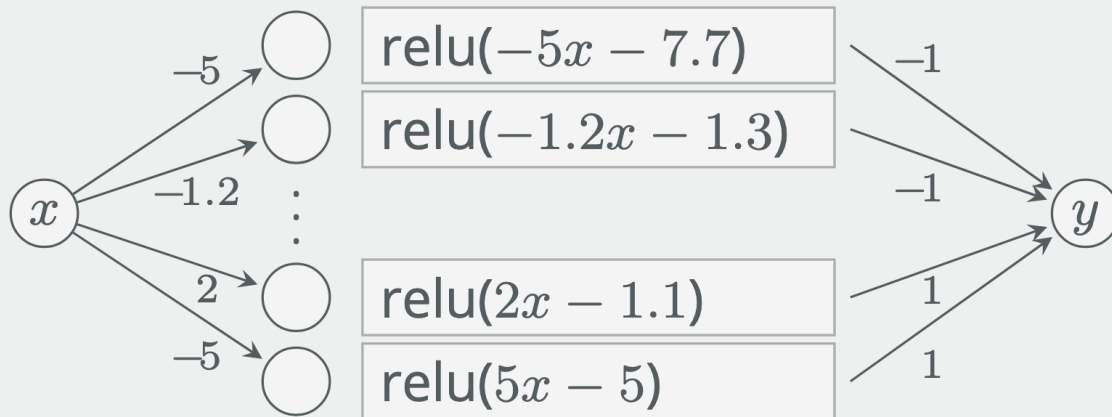
universal approximation theorem



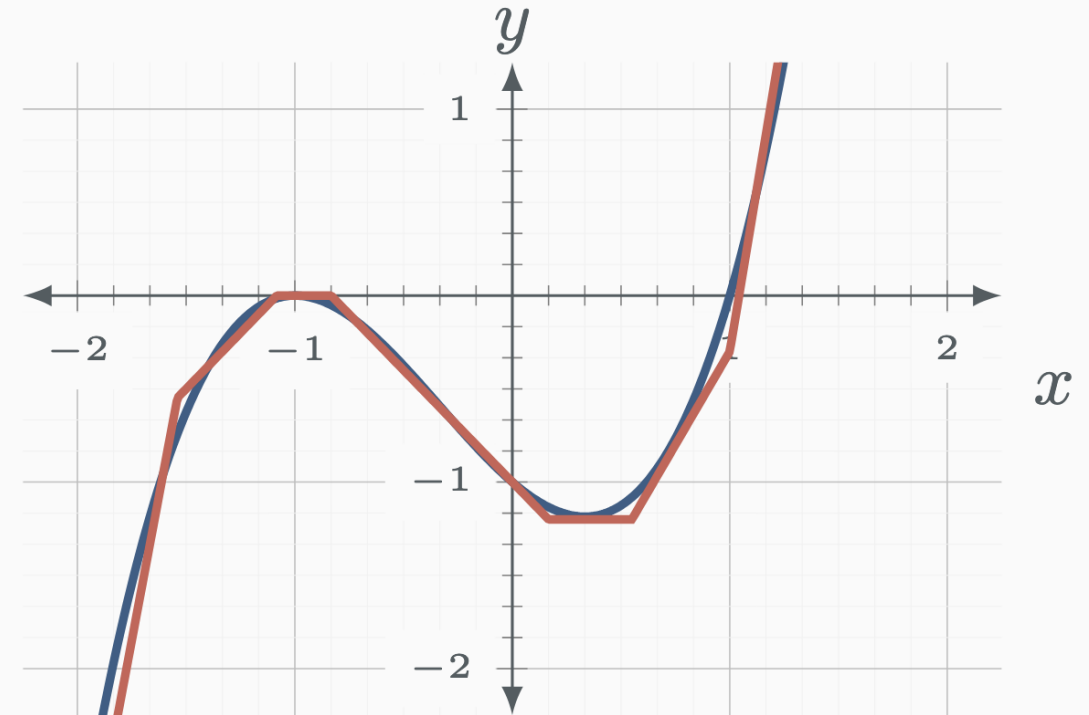
Theorem: universal function approximator

Arbitrary width

A network with a single hidden layer, containing a finite number of neurons, can approximate any continuous function under mild assumptions.



original function $f(x) = x^3 + x^2 - x - 1$



$$f(x) = -r(-5x - 7.7) - r(-1.2x - 1.3) - r(1.2x + 1) + r(1.2x - 0.2) + r(2x - 1.1) + r(5x - 5)$$

Theorem: universal function approximator

Arbitrary depth (fixed width)

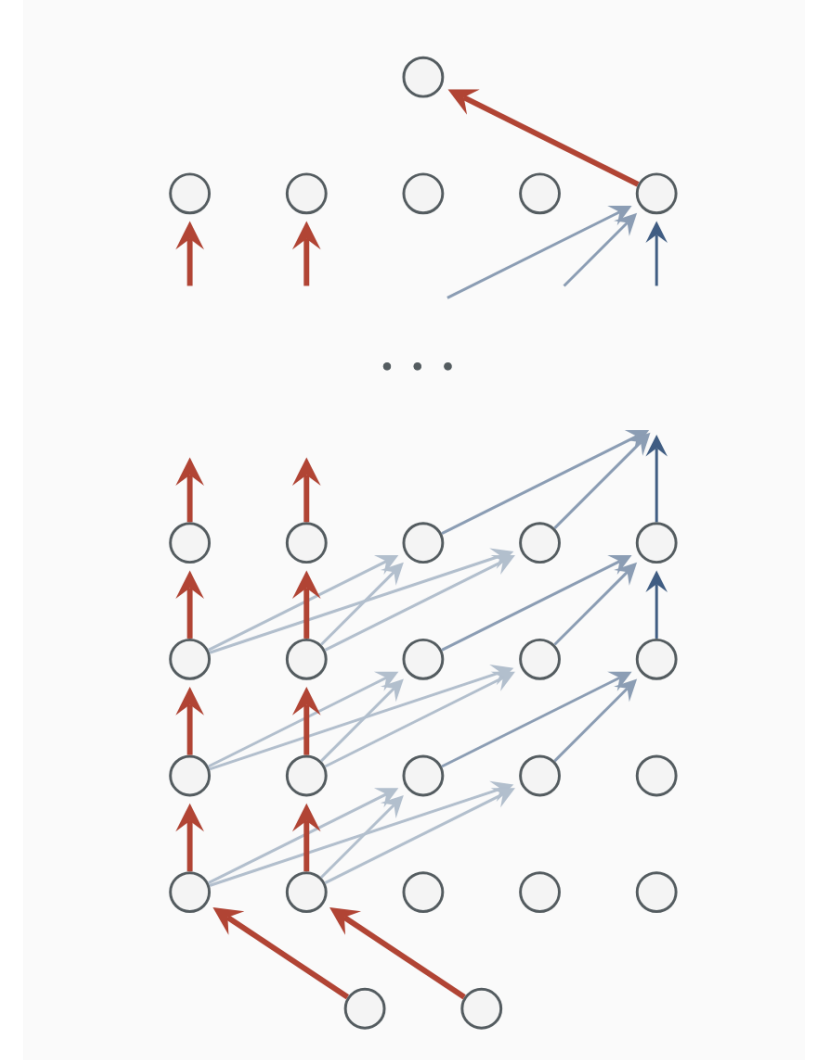
Does the theorem still hold for fixed width and arbitrary depth? **Yes!**

For a network of n inputs and m outputs, [1] show universal approximation holds true for:

- width $n + m + 2$ for almost any activation function
- width $n + m + 1$ for most activation functions

[1] Kidger et al., 2020

<https://www.youtube.com/watch?v=6-SORkCLYPM>





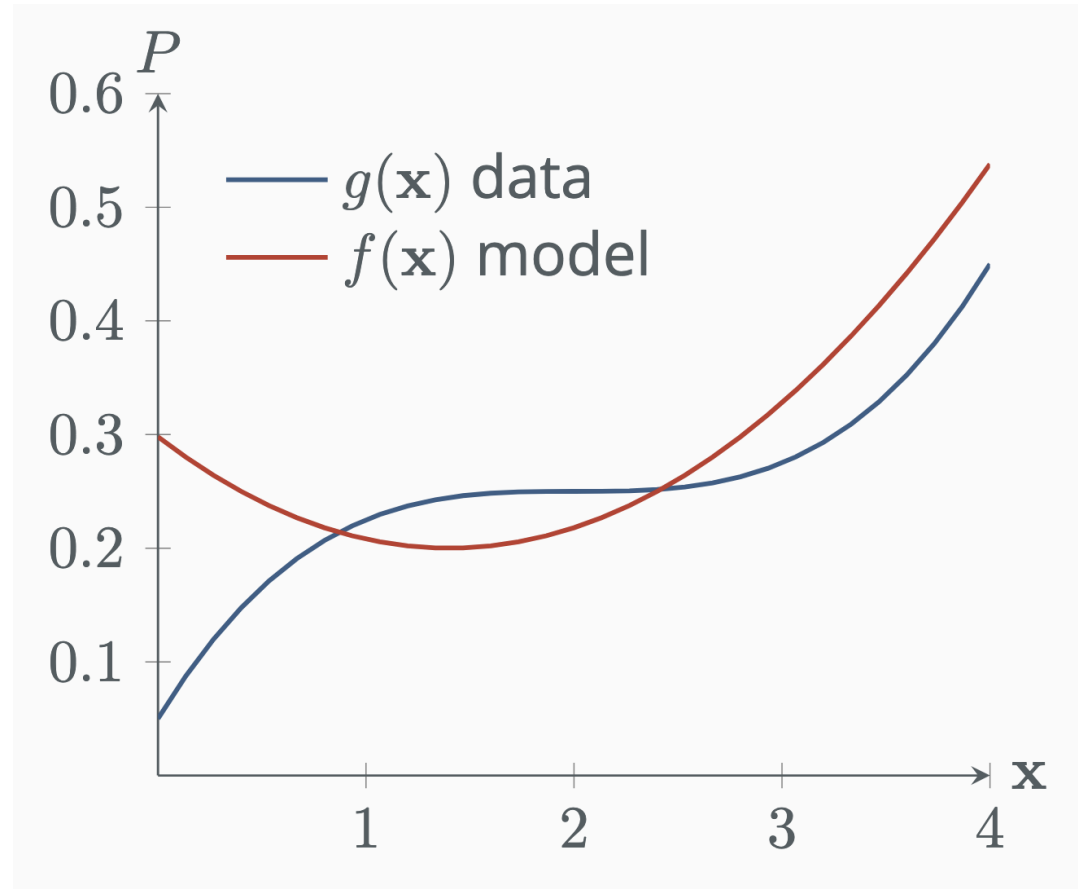
Finding a model

Lets illustrate this in 1D, but try to imagine it in ND . Given the target probability distribution of our data

$$g(\mathbf{x}) = P(X, Y)$$

we want to find (design) a model $f(\mathbf{x}; \theta)$ with parameters θ and optimise θ such that

$$f(\mathbf{x}; \theta) = g(\mathbf{x})$$





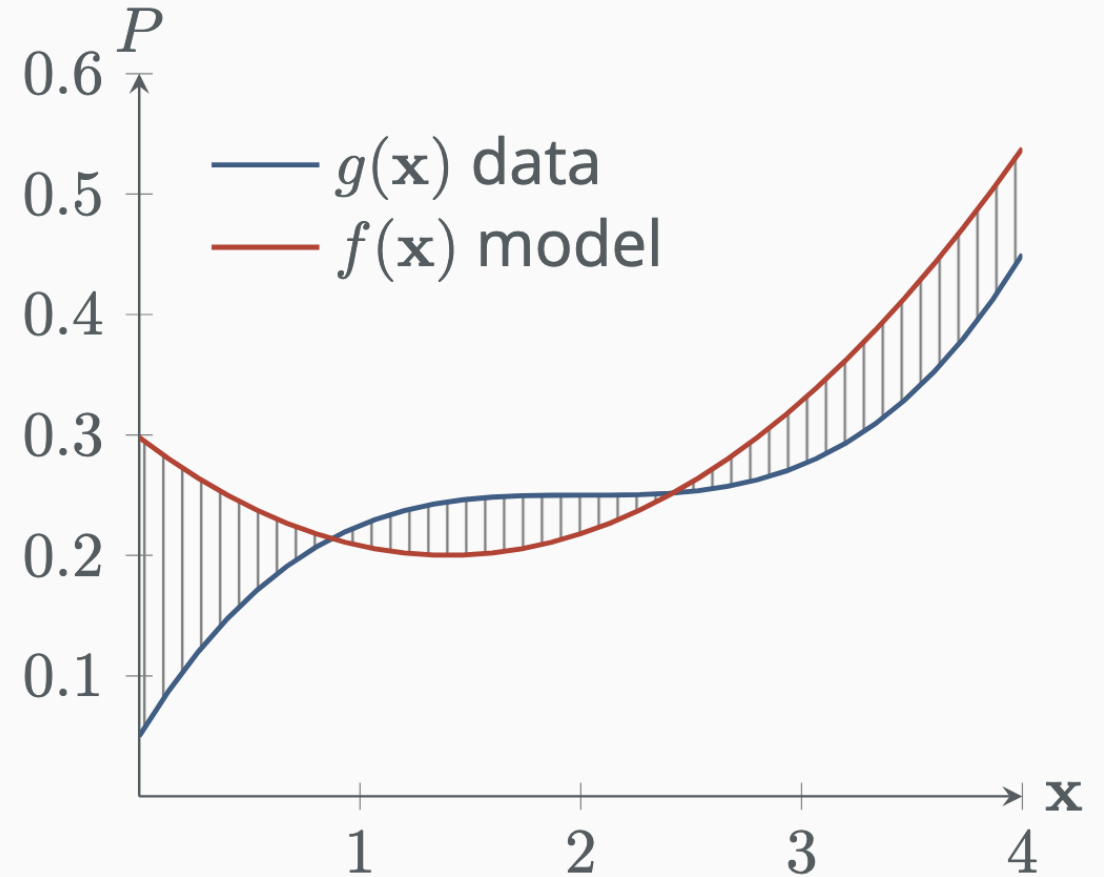
Optimising the model

The total error is therefore the entire area. Modifying the parameters θ will cause the area to change, where we want to find

$$\hat{\theta} = \arg \min_{\theta} \int \mathcal{L}(f(\mathbf{x}; \theta), g(\mathbf{x})) d\mathbf{x}.$$

where $\mathcal{L}$ is a 'loss function', e.g. a 0-1 loss function $\mathcal{L}(\hat{x}, x) = \mathbb{I}(\hat{x} \neq x)$ or a mean squared error loss.

The solution is a function f that has the capacity to exactly represent $g(\mathbf{x})$





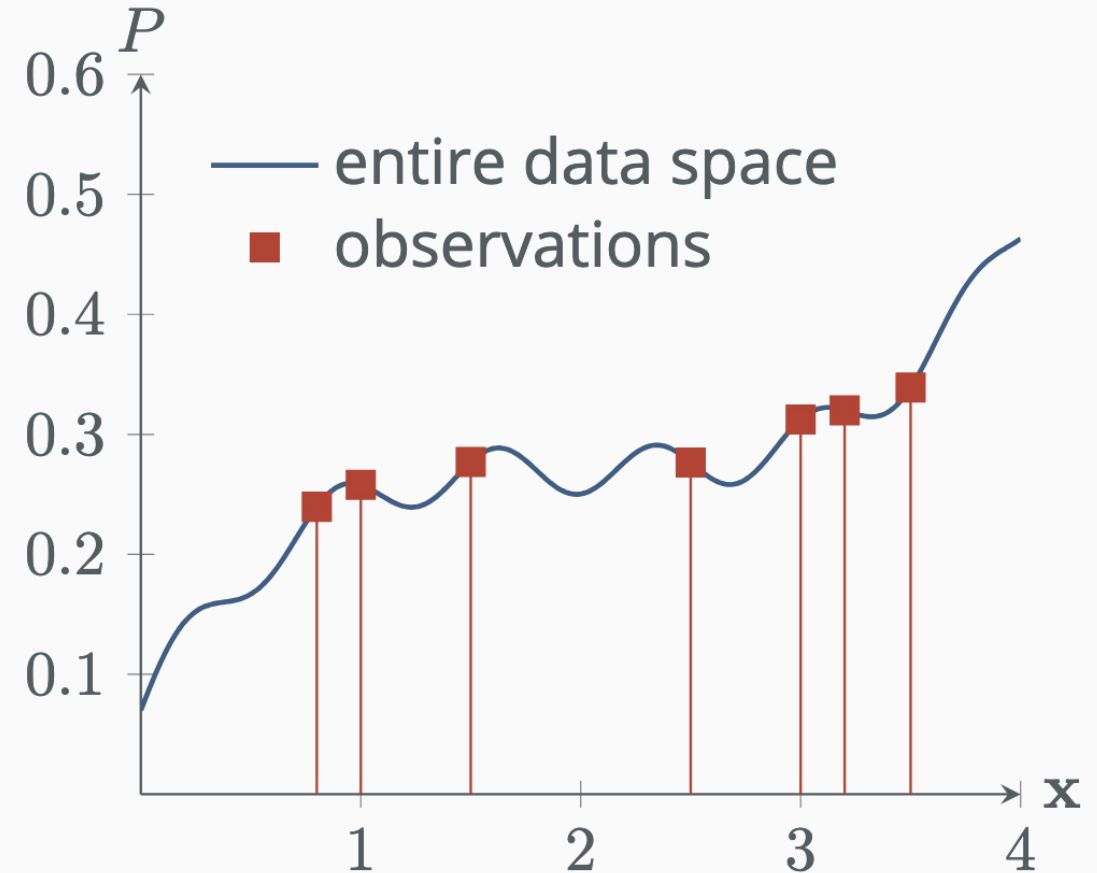
The generalisation problem

However there's a big problem:

In practice, we can't observe all of $g(\mathbf{x})$

This means:

1. We don't know how smooth the function is between the observations
2. Noise can be difficult to interpret
3. Optimisation is highly sensitive to the sampling process

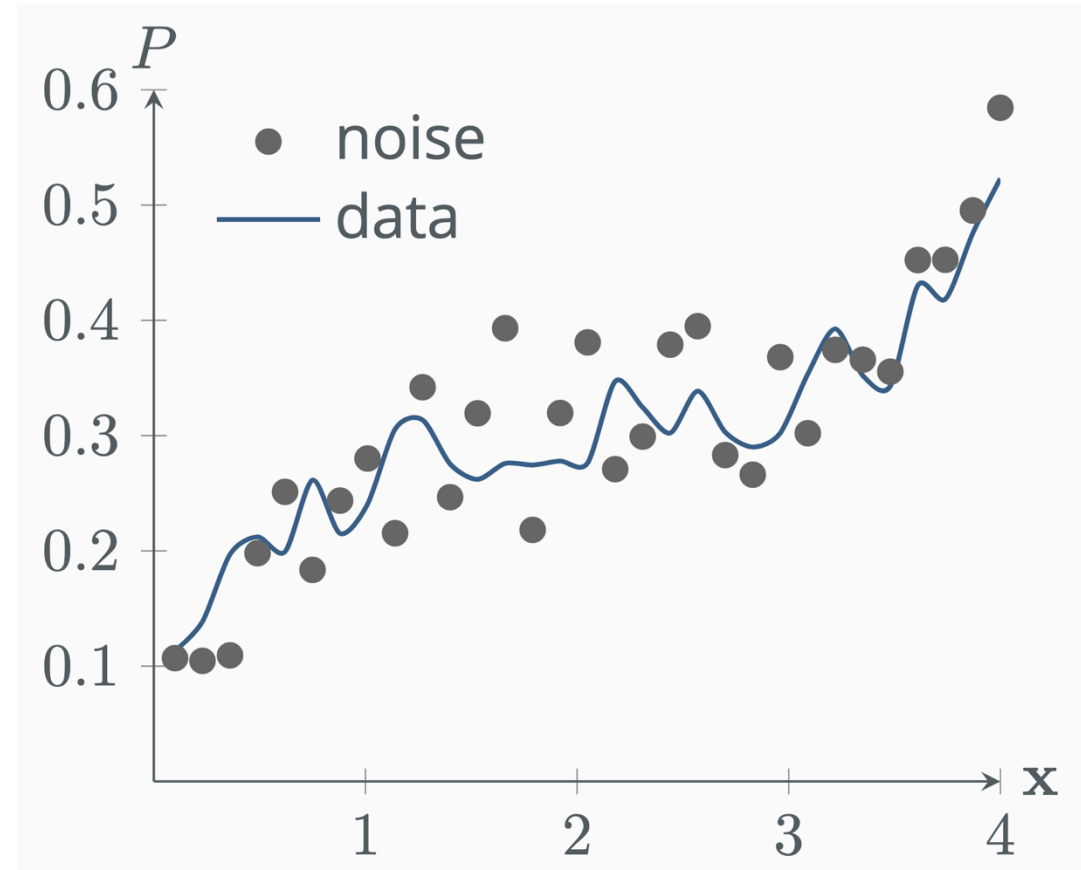




Noise and regularisation

Given that the shape of the distribution outside of the observations is unknown, it is easy to overfit to noise, especially when you have limited data.

We can often regularise our function to be invariant to this.

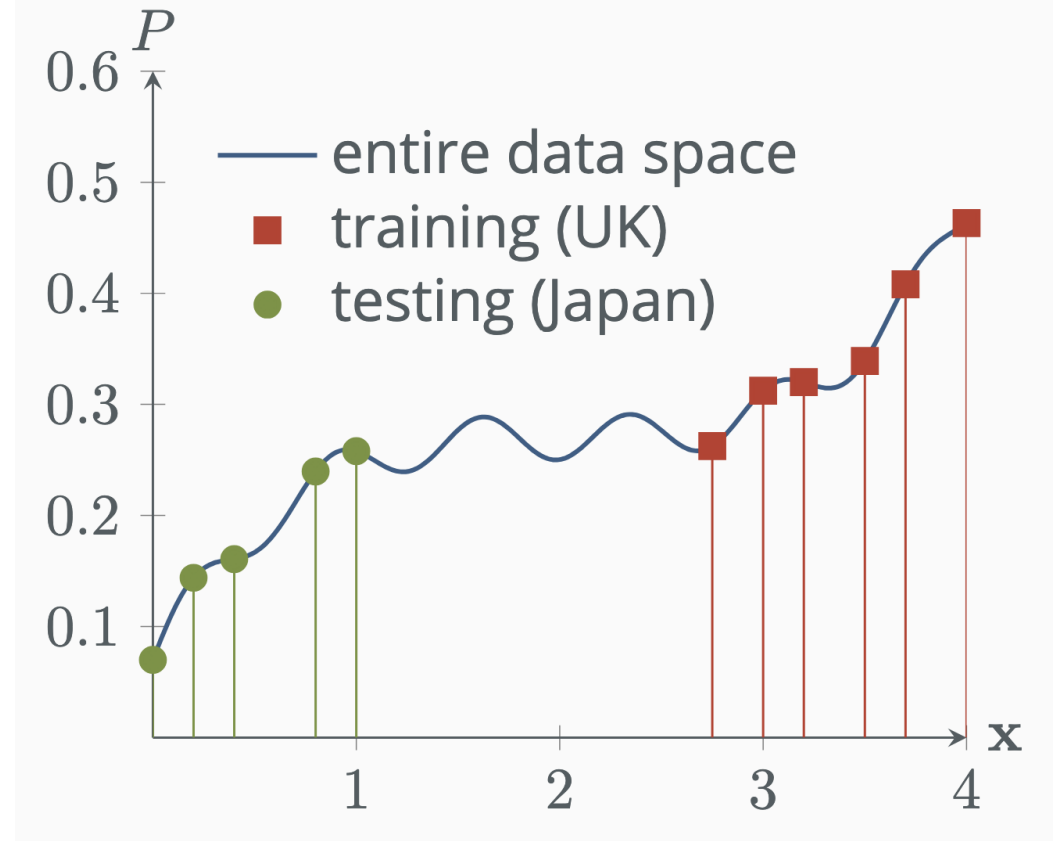




Out-of-distribution data

Usually the training dataset collection process draws samples from the data space in a way that is not **independent and identically distributed** (abbreviated i.i.d.) to the expected testing (operational) conditions of the model.

Sampling data in a way that is representative of the task/testing/operational distribution is extraordinarily difficult to do properly. It is often a worthwhile investment though!



Similarity measures

How can we evaluate the generalisation of unsupervised generative models?

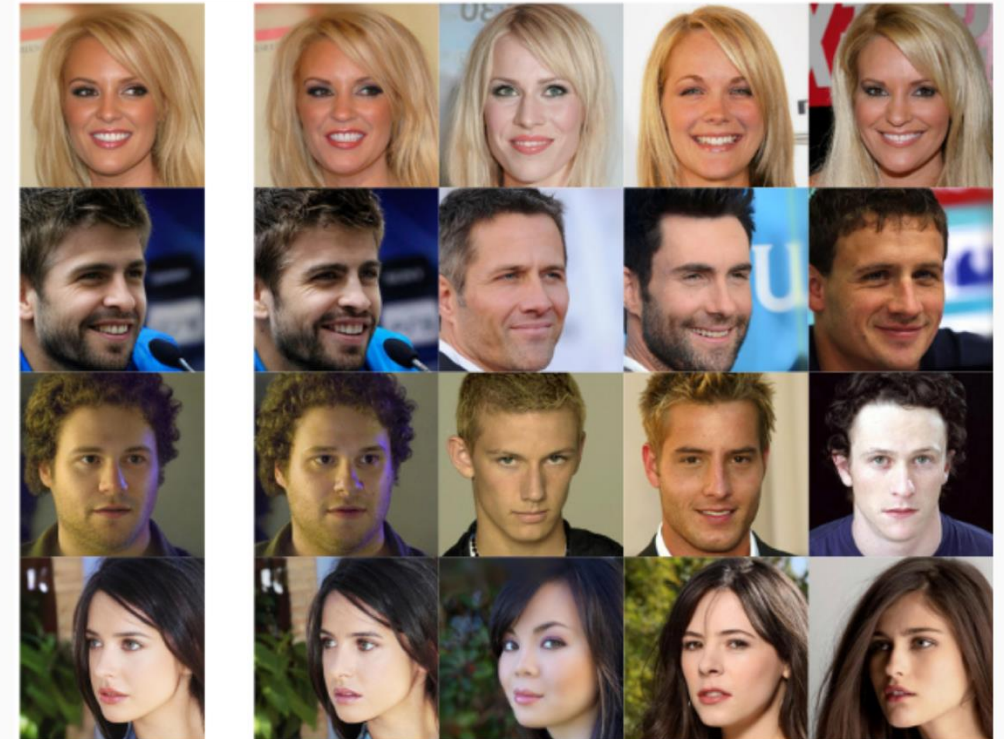
Deep pre-trained network activations work surprisingly well (e.g. FID, LPIPS [2] for perceptual similarity between images):

```
import lpips
distance_fn = lpips.LPIPS(net="alex")
```

[2] Zhang et al., 2018

Link to code example:

https://github.com/samb-t/unleashing-transformers/blob/master/experiments/calc_nearest_neighbours.py



Model samples (leftmost column) and nearest neighbours in training set (LPIPS distance [2]) increasing to right.

Generalisation Theory

no free lunch theorem and Occam's razor

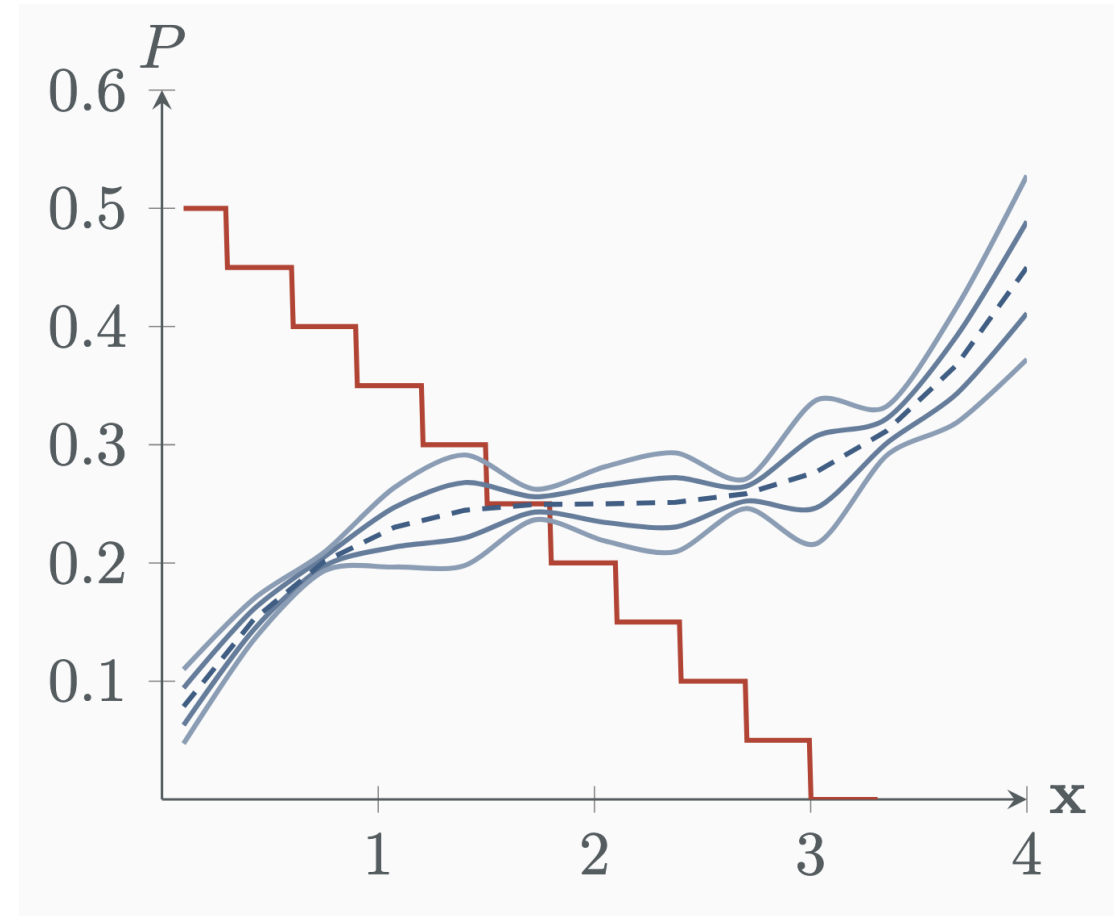


Definition: no free lunch theorem

We can use methods such as cross validation to empirically choose the best method for our particular problem. However, there is no universally best model — this is sometimes called the no free lunch theorem [3].

Definition: Occam's razor

'Prefer the simplest hypothesis $\mathcal{H}$ that fits the data.' In the case of deep learning, this implies the smoothest function that fits the data.



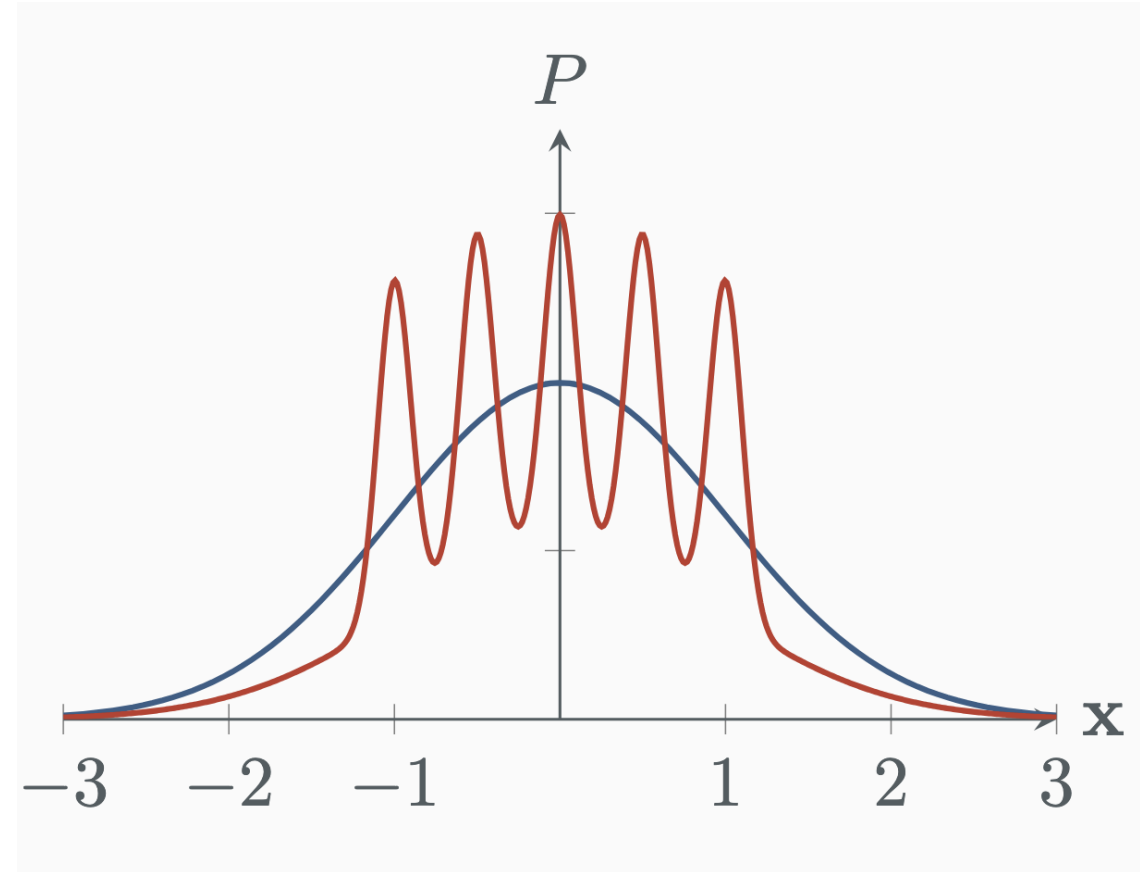


Example: problematic densities

In a more complex setting, we may have a density such as the 'Bart Simpson' density, as Nando de Freitas likes to call it

$$p(x) = \frac{1}{2} \phi(x; 0, 1) + \frac{1}{10} \sum_{j=0}^4 \phi(x; (j/2) - 1, 1/10)$$

where ϕ is the normal density with mean μ and standard deviation σ . This density cannot be sufficiently estimated with a normal distribution, as the result is over-smoothed (blue).



Generalisation Theory

increasing capacity to double descent



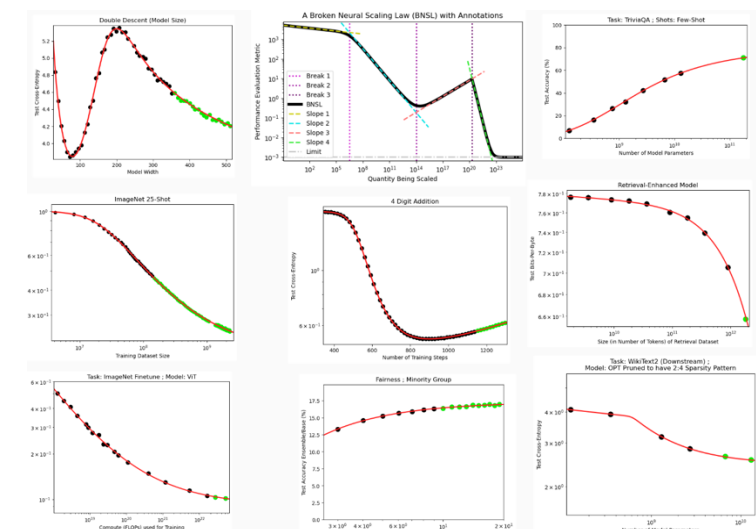
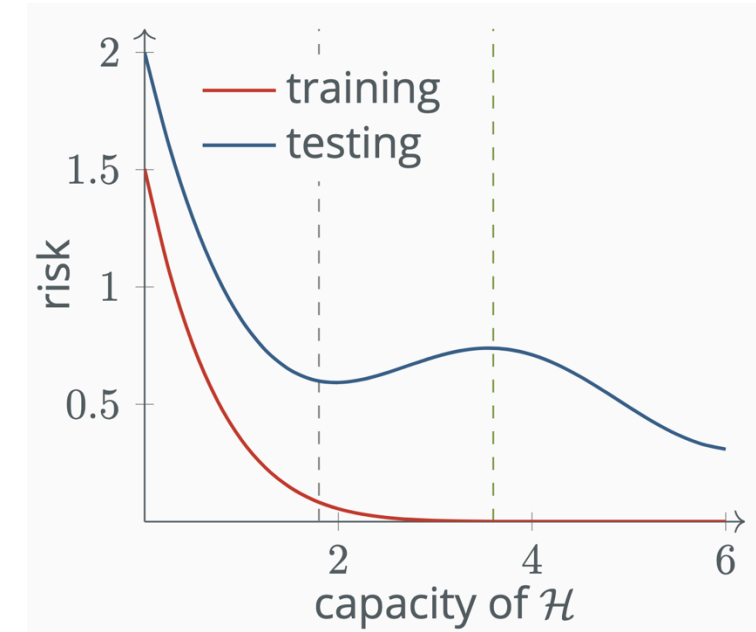
Definition: double descent

Traditionally, we know that increasing the parameters lowers the bias (fitting), but the variance (test risk) will eventually reach a 'sweet spot' (first dashed line) and start to increase again.

The full story has a double descent curve [4], as higher capacity functions past the interpolation threshold (second dashed line) lead again to smoother fitting (Occam's razor).

[4] Belkin et al., 2019

This can be predicted:



What we learned today!



Summary

In summary:

- deep learning isn't magic
- there's lots of formal results
- understand the various trade-offs (bias-variance, linear transformers, modelling trilemma)
- categorise the contributions into the three spaces (data, function, modelling) at the end of lecture 1
- ...predictions can be made well by the *very few* researchers who make predictions...